HDOC Day-53

Write a C program that reads the size and elements of a variable-length integer array, nums[]. Pass the array to a function named product_except_self() using its address. Use pointers to pass the array between the calling function and product_except_self(). The function product_except_self() must:

- Create an output array, answer[], of the same size as nums[].
- Store in answer[i] the product of all elements of nums[] except nums[i].
- Ensure that the product does not include nums[i].
- Display the elements of answer[] in their original order, separated by commas.
- Avoid using division to calculate the products.
- Use the fact that the product of every prefix and suffix fits within a 32-bit integer.

Example 1:

Array: 1 2 3 4

Output: 24, 12, 8, 6

Example 2:

Array: -1 1 0 -3 3

Output: 0, 0, 9, 0, 0

Algorithm: Product of Array Except Self

1. **Start.**
2. Read the size of the array n.
3. Declare the integer array nums[n].
4. Read n elements into nums[].
5. Pass the address of nums[] and n to the function product_except_self().
6. Inside product_except_self():
    - Declare an array answer[n].
    - Initialize prefix = 1.

7. Traverse the array from **left to right**:
    - Store prefix in answer[i].
    - Update prefix = prefix * nums[i].
8. Initialize suffix = 1.
9. Traverse the array from **right to left**:
    - Update answer[i] = answer[i] * suffix.
    - Update suffix = suffix * nums[i].
10. Display all elements of answer[] separated by commas.
11. **Stop.**

- C Program:
- #include <stdio.h>
-
- void product_except_self(int *nums, int size)
- {
- int answer[size];
- int prefix = 1;
- int suffix = 1;
-
- // Store prefix products in answer[]
- for (int i = 0; i < size; i++)
- {
- answer[i] = prefix;
- prefix *= *(nums + i);
- }
-
- // Multiply by suffix products
- for (int i = size - 1; i >= 0; i--)
- {
- answer[i] *= suffix;
- suffix *= *(nums + i);
- }
-
- // Display answer[]
- for (int i = 0; i < size; i++)
- {

```
        printf("%d", answer[i]);

        if (i < size - 1)
            printf(", ");
    }

    printf("\n");
}

int main()
{
    int size;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int nums[size];

    printf("Enter %d integers: ", size);

    for (int i = 0; i < size; i++)
    {
        scanf("%d", nums + i);
    }

    // Pass the array using its address
    product_except_self(nums, size);

    return 0;
}
```

Test Case Table:

Test Case	Size	Input array	Expected Output	Purpose
1	4	1 2 3 4	24 , 12 , 8 , 6	Normal positive integers

2	5	-1 1 0 -3 3	0 , 0 , 9 , 0 , 0	Contain one zero and negative values
3	4	2 3 4 5	60 , 40 , 30 , 24	Positive integers
4	3	1 1 1	1 , 1 , 1	All elements are 1
5	4	0 2 0 4	0 , 0 , 0 , 0	Contains multiple zeroes
6	3	-1 -2 -3	6 , 3 , 2	All negative integers
7	2	5 10	10 , 5	Small array
8	5	1 2 3 4 5	120 , 60 , 40 , 30 , 24	Larger sequential input

Compilation and Execution:

The following program has been compiled and executed successfully.

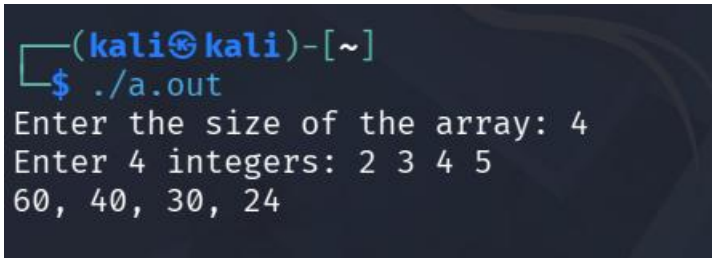
Test Case 1:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 4
Enter 4 integers: 1 2 3 4
24, 12, 8, 6
```

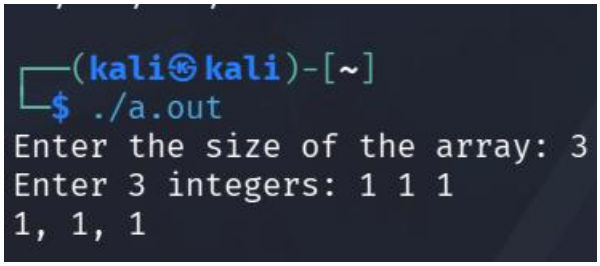
Test Case 2:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 5
Enter 5 integers: -1 1 0 -3 3
0, 0, 9, 0, 0
```

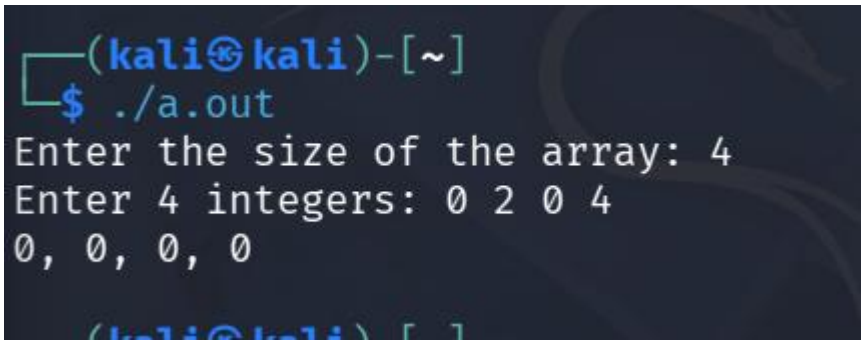
Test Case 3:



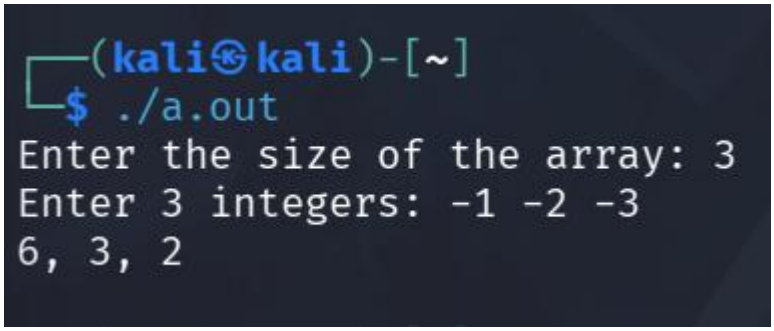
Test Case 4:



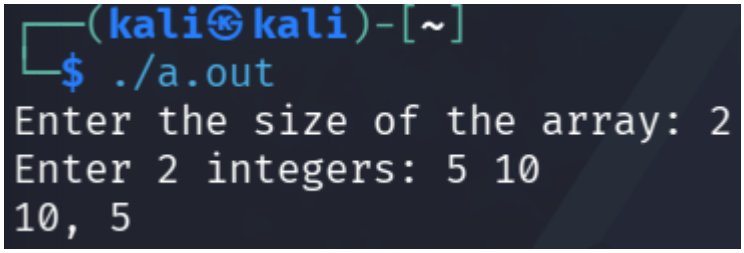
Test Case 5:



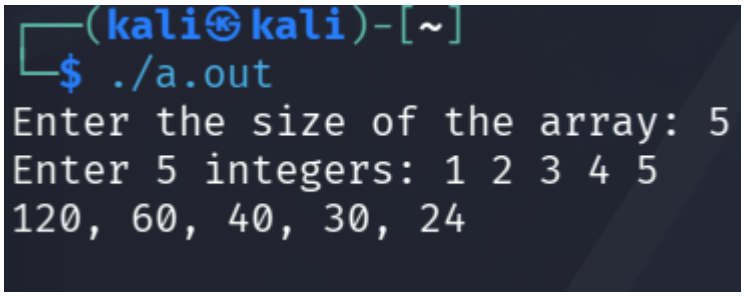
Test Case 6:



Test Case 7:


(kali㉿kali)-[~]
$ ./a.out
Enter the size of the array: 2
Enter 2 integers: 5 10
10, 5

Test Case 8:


(kali㉿kali)-[~]
$ ./a.out
Enter the size of the array: 5
Enter 5 integers: 1 2 3 4 5
120, 60, 40, 30, 24